**Question 1** | Correct Mark 1.00 out of 1.00

Convert the following algorithm into a program and find its time complexity using the counter method.

```
void function (int n)
{
    int i= 1;
```

```
    int s =1;
```

```
    while(s <= n)
    {
         i++;
         s += i;
     }
}
```

**Note:** No need of counter increment for declarations and scanf() and count variable printf() statements.

**Input:**

A positive Integer n

**Output:**

Print the value of the counter variable

**For example:**

| Input | Result |
|---|---|
| 9 | 12 |

**Answer:** (penalty regime: 0 %)

```
#include<stdio.h>
void function(int n){
    int count=0;
    count++;
    int i=1;
    count++;
    int s=1;
    count++;
    while(s<=n){
    count++;
    i++;
    count++;
    s+=i;
    count++;
    }
    printf("%d",count);
}
int main(){
int n;
scanf("%d",&n);
function(n);
}
```

| | Input | Expected | Got | |
|---|---|---|---|---|
| ✔ | 9 | 12 | 12 | ✔ |
| ✔ | 4 | 9 | 9 | ✔ |

Passed all tests! ✔

Correc t

Marks for this submission: 1.00/1.00.

Back to Course

**Question 1** | Correct Mark 1.00 out of 1.00

Convert the following algorithm into a program and find its time complexity using the counter method.

```
void func(int n)
{
    if(n==1)
    {
      printf("*");
    }
    else
    {
     for(int i=1; i<=n; i++)
     {
       for(int j=1; j<=n; j++)
       {
          printf("*");
          printf("*");
          break;
       }
     }
   }
 }
```

**Note:** No need of counter increment for declarations and scanf() and count variable printf() statements.

**Input:**

A positive Integer n

**Output:**

Print the value of the counter variable

**Answer:** (penalty regime: 0 %)

```
#include<stdio.h>
void func(int n)
{
int count=0;
count++;
if(n==1)
{
//printf("");
}
else
{
for(int i=1; i<=n; i++)
{
count++;
count++;
for(int j=1; j<=n; j++)
{
count++;
//printf("");
//printf("*");
count++;
break;
}
count++;
}
count++;
}
printf("%d",count);
}
int main(){
int n;
scanf("%d",&n);
func(n);
}
```

| | Input | Expected | Got | |
|---|---|---|---|---|
| | 2 | 12 | 12 | |
| | 1000 | 5002 | 5002 | |
| | 143 | 717 | 717 | |

Passed all tests!

Correc t

Marks for this submission: 1.00/1.00.

Back to Course

**Question 1** | Correct Mark 1.00 out of 1.00

Convert the following algorithm into a program and find its time complexity using counter method.

```
Factor(num) {
{
   for (i = 1; i <= num;++i)
   {
    if (num % i== 0)
       {
         printf("%d ", i);
       }
    }
 }
```

**Note:** No need of counter increment for declarations and scanf() and counter variable printf() statement.

**Input:**

A positive Integer n

**Output:**

Print the value of the counter variable

**Answer:**

```c
#include<stdio.h>
void Factor(int num) {
    int count=0;
    for (int i = 1; i <= num;++i){
    count++;
    count++;
        if (num % i== 0){
            count++;

            //printf("%d ", i);
        }
    }
    count++;
    printf("%d",count);
}
int main(){
int num;
scanf("%d",&num);
Factor(num);
}
```

| | Input | Expected | Got | |
|---|---|---|---|---|
| ✔ | 12 | 31 | 31 | ✔ |
| ✔ | 25 | 54 | 54 | ✔ |
| ✔ | 4 | 12 | 12 | ✔ |

Passed all tests! ✔

Correct

Marks for this submission: 1.00/1.00.

Back to Course

**Question 1** | Correct Mark 1.00 out of 1.00

Convert the following algorithm into a program and find its time

complexity using counter method.

```
void function(int n)
{
    int c= 0;
    for(int i=n/2; i<n; i++)
        for(int j=1; j<n; j = 2 * j)
            for(int k=1; k<n; k = k * 2)
                c++;
}
```

**Note:** No need of counter increment for declarations and scanf() and count variable printf() statements.

**Input:**

A positive Integer n

**Output:**

Print the value of the counter variable

**Answer:**

```
#include<stdio.h>
void function(int n)
{
    int count=0;
    count++;
    int c= 0;
    count++;
    for(int i=n/2; i<n; i++){
        count++;
        count++;
        for(int j=1; j<n; j = 2 * j){
            count++;
            count++;
            for(int k=1; k<n; k = k * 2){
                count++;
                count++;
                c++;
            }
        }
    }
    printf("%d",count);

}
int main(){
    int n;
    scanf("%d",&n);
    function(n);
}
```

| | Input | Expected | Got | |
|---|---|---|---|---|
| ✔ | 4 | 30 | 30 | ✔ |
| ✔ | 10 | 212 | 212 | ✔ |

Passed all tests! ✔

Correct

Marks for this submission: 1.00/1.00.

Back to Course

**Question 1** | Correct Mark 1.00 out of 1.00

Convert the following algorithm into a program and find its time complexity using counter method.

```
void reverse(int n)
{
   int rev = 0, remainder;
   while (n != 0)
    {
        remainder = n % 10;
        rev = rev * 10 + remainder;
        n/= 10;

    }
print(rev);
}
```

**Note:** No need of counter increment for declarations and scanf() and count variable printf() statements.

**Input:**

A positive Integer n

**Output:**

Print the value of the counter variable

**Answer:**

```c
#include<stdio.h>

void reverse(int n)
{
    int count=0;
    count++;
    int rev = 0;
    count++;
    int remainder;
   while (n != 0)
    {
        count++;
        remainder = n % 10;
        count++;
        rev = rev * 10 + remainder;
        count++;
        n/= 10;
        count++;
    }
    count++;
//print(rev);
printf("%d",count);
}
int main(){
    int n;
    scanf("%d",&n);
    reverse(n);
}
```

| | Input | Expected | Got | |
|---|---|---|---|---|
| ✔ | 12 | 11 | 11 | ✔ |
| ✔ | 1234 | 19 | 19 | ✔ |

Passed all tests! ✔

Correc t

Marks for this submission: 1.00/1.00.

Back to Course

**Question 1** | Correct Mark 1.00 out of 1.00

Write a program to take value V and we want to make change for V Rs, and we have in finite supply of each of the denominations in Indian currency, i.e., we have in finite supply of { 1, 2, 5, 10, 20, 50, 100, 500, 1000} valued coins/notes, what is the minimum number of coins and/or notes needed to make the change.

Input Format:

Take an integer from stdin.

Output Format:

print the integer which is change of the number.

Example Input :

64

Output:

4

Explanaton:

We need a 50 Rs note and a 10 Rs note and two 2 rupee coins.

**Answer:** (penalty regime: 0 % )

```
#include<stdio.h>
int main(){
int v,count=0;
int denominations[]={1000,500,100,50,20,10,5,2,1};
scanf("%d",&v);
for(int i=0;i<9;i++){
count+=v/denominations[i];
v=v%denominations[i];
}
printf("%d",count);
return 0;
}
```

| | Input | Expected | Got | |
|---|---|---|---|---|
| ? | 49 | 5 | 5 | ? |

Passed all tests! ?

Correct

Marks for this submission: 1.00/1.00.

Back to Course

**Question 1** | Correct Mark 1.00 out of 1.00

Assume you are an awesome parent and want to give your children some cookies. But, you should give each child at most one cookie.

Each child i has a greed factor g[i], which is the minimum size of a cookie that the child will be content with; and each cookie j has a size s[j]. If s[j] >= g[i], we can assign the cookie j to the child i, and the child i will be content. Your goal is to maximize the number of your content children and output the maximum number.

**Example 1:**

**Input:**

3

1 2 3

2

1 1

**Output:**

1

Explanation: You have 3 children and 2 cookies. The greed factors of 3 children are 1, 2, 3.

And even though you have 2 cookies, since their size is both 1, you could only make the child whose greed factor is 1 content.

You need to output 1.

**Constraints:**

1 <= g.length <= 3 * 10^4

0 <= s.length <= 3 * 10^4

1 <= g[i], s[j] <= 2^31 - 1

**Answer:** (penalty regime: 0 %)

```c
#include <stdio.h>

int main() {
    int n, m;
    scanf("%d", &n);

    int g[n];
    for (int i = 0; i < n; i++)
        scanf("%d", &g[i]);

    scanf("%d", &m);

    int s[m];
    for (int i = 0; i < m; i++)
        scanf("%d", &s[i]);

    for (int i = 0; i < n - 1; i++) {
        for (int j = i + 1; j < n; j++) {
            if (g[i] > g[j]) {
                int temp = g[i];
                g[i] = g[j];
                g[j] = temp;
            }
        }
    }

    for (int i = 0; i < m - 1; i++) {
        for (int j = i + 1; j < m; j++) {
            if (s[i] > s[j]) {
                int temp = s[i];
                s[i] = s[j];
                s[j] = temp;
            }
        }
    }

    int i = 0, j = 0, count = 0;

    while (i < n && j < m) {
        if (s[j] >= g[i]) {
            count++;
            i++;
            j++;
        } else {
            j++;
        }
```

```
47      }
48
49      printf("%d", count);
50
51      return 0;
52  }
```

| | Input | Expected | Got | |
|---|---|---|---|---|
| ✓ | 2<br><br>1 2<br><br>3<br><br>1 2 3 | 2 | 2 | ✓ |

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.

Back to Course

**Question 1** | Correct Mark 1.00 out of 1.00

A person needs to eat burgers. Each burger contains a count of calorie. After eating the burger, the person needs to run a distance to burn out his calories.

If he has eaten $i$ burgers with c calories each, then he has to run at least $3^i * c$ kilometers to burn out the calories. For example, if he ate 3 burgers with the count of calorie in the order: [1, 3, 2], the kilometers he needs to run are $(3^0 * 1) + (3^1 * 3) + (3^2 * 2) = 1 + 9 + 18 = 28$.

But this is not the minimum, so need to try out other orders of consumption and choose the minimum value. Determine the minimum distance he needs to run. Note: He can eat burger in any order and use an efficient sorting algorithm.Apply greedy approach to solve the problem.

**Input Format**

First Line contains the number of burgers

Second line contains calories of each burger which is n space-separate integers

**Output Format**

Print: Minimum number of kilometers needed to run to burn out the calories

**Sample Input**

```
3
5 10 7
```

**Sample Output**

```
76
```

**For example:**

| Test | Input | Result |
|---|---|---|
| Test Case 1 | 3<br>1 3 2 | 18 |

**Answer:** (penalty regime: 0 %)

```
#include<stdio.h>
#include<math.h>
int main(){
    int n;
    scanf("%d",&n);
    int arr[n];
    for(int i=0;i<n;i++){
        scanf("%d",&arr[i]);
    }
    for (int i = 0; i < n - 1; i++) {
        for (int j = 0; j < n - i - 1; j++) {
            if (arr[j] < arr[j + 1]) {
                int temp = arr[j];
                arr[j] = arr[j + 1];
                arr[j + 1] = temp;
            }
        }
    }
    int s=0;
    for(int i=0;i<n;i++){
        s+=(pow(n,i))*arr[i];
    }
    printf("%d",s);
}
```

| | Test | Input | Expected | Got | |
|---|---|---|---|---|---|
| ✔ | Test Case 1 | 3<br>1 3 2 | 18 | 18 | ✔ |
| ✔ | Test Case 2 | 4<br>7 4 9 6 | 389 | 389 | ✔ |
| ✔ | Test Case 3 | 3<br>5 10 7 | 76 | 76 | ✔ |

Passed all tests! ✔

Correct

Marks for this submission: 1.00/1.00.

Back to Course

**Question 1** | Correct Mark 1.00 out of 1.00

Given an array of N integer, we have to maximize the sum of arr[i] * i, where i is the index of the element (i = 0, 1, 2, ..., N).Write an algorithm based on Greedy technique with a Complexity O(nlogn).

Input Format:

First line specifies the number of elements-n

The next n lines contain the array elements.

Output Format:

Maximum Array Sum to be printed.

Sample Input:

5

2 5 3 4 0

Sample output:

40

**Answer:** (penalty regime: 0 %)

```c
#include <stdio.h>
#include <stdlib.h>

int compare(const void *a, const void *b) {
    return (*(int *)a - *(int *)b);
}

int main() {
    int n;
    scanf("%d", &n);

    int a[n];

    for (int i = 0; i < n; i++)
        scanf("%d", &a[i]);

    qsort(a, n, sizeof(int), compare);

    long long sum = 0;

    for (int i = 0; i < n; i++) {
        sum += (long long)a[i] * i;
    }

    printf("%lld", sum);

    return 0;
}
```

| | Input | Expected | Got | |
|---|---|---|---|---|
| ✔ | 5<br>2<br>5<br>3<br>4<br>0 | 40 | 40 | ✔ |

| | Input | Expected | Got | |
|---|---|---|---|---|
| ✔ | 10<br>2<br>2<br>2<br>4<br>4<br>3<br>3<br>5<br>5<br>5 | 191 | 191 | ✔ |
| ✔ | 2<br>45<br>3 | 45 | 45 | ✔ |

Passed all tests! ✔

Correct

Marks for this submission: 1.00/1.00.

Back to Course

**Question 1** | Correct Mark 1.00 out of 1.00

Given two arrays array_One[] and array_Two[] of same size N. We need to first rearrange the arrays such that the sum of the product of pairs( 1 element from each) is minimum. That is SUM (A[i] * B[i]) for all i is minimum.

**For example:**

| Input | Result |
|---|---|
| 3<br>1<br>2<br>3<br>4<br>5<br>6 | 28 |

**Answer:** (penalty regime: 0 %)

```c
#include <stdio.h>

int main() {
    int n, i, j, temp;
    scanf("%d", &n);

    int a[n], b[n];

    for (i = 0; i < n; i++)
        scanf("%d", &a[i]);

    for (i = 0; i < n; i++)
        scanf("%d", &b[i]);

    for (i = 0; i < n - 1; i++) {
        for (j = i + 1; j < n; j++) {
            if (a[i] > a[j]) {
                temp = a[i];
                a[i] = a[j];
                a[j] = temp;
            }
        }
    }

    for (i = 0; i < n - 1; i++) {
        for (j = i + 1; j < n; j++) {
            if (b[i] < b[j]) {
                temp = b[i];
                b[i] = b[j];
                b[j] = temp;
            }
        }
    }

    long long sum = 0;

    for (i = 0; i < n; i++)
        sum += (long long)a[i] * b[i];

    printf("%lld", sum);

    return 0;
}
```

| | Input | Expected | Got | |
|---|---|---|---|---|
| ✔ | 3<br>1<br>2<br>3<br>4<br>5<br>6 | 28 | 28 | ✔ |
| ✔ | 4<br>7<br>5<br>1<br>2<br>1<br>3<br>4<br>1 | 22 | 22 | ✔ |
| ✔ | 5<br>20<br>10<br>30<br>10<br>40<br>8<br>9<br>4<br>3<br>10 | 590 | 590 | ✔ |

Passed all tests! ✔

Correct

Marks for this submission: 1.00/1.00.

Back to Course

**Question 1** | Correct Mark 1.00 out of 1.00

**Problem Statement**

Given an array of 1s and 0s this has all 1s first followed by all 0s. Aim is to find the number of 0s. Write a program using Divide and Conquer to Count the number of zeroes in the given array.

Input Format

First Line Contains Integer m – Size of array

Next m lines Contains m numbers – Elements of an array

Output Format

First Line Contains Integer – Number of zeroes present in the given array.

**Answer:** (penalty regime: 0 %)

```c
#include <stdio.h>

int findFirstZero(int a[], int low, int high) {
    if (low > high)
        return -1;

    int mid = (low + high) / 2;

    if (a[mid] == 0) {
        if (mid == 0 || a[mid - 1] == 1)
            return mid;

        return findFirstZero(a, low, mid - 1);
    }

    return findFirstZero(a, mid + 1, high);
}

int main() {
    int n;
    scanf("%d", &n);

    int a[n];

    for (int i = 0; i < n; i++)
        scanf("%d", &a[i]);

    int index = findFirstZero(a, 0, n - 1);

    if (index == -1)
        printf("0");
    else
        printf("%d", n - index);

    return 0;
}
```

| | Input | Expected | Got | |
|---|---|---|---|---|
| ✔ | 5<br>1<br>1<br>1<br>0<br>0 | 2 | 2 | ✔ |

| | Input | Expected | Got | |
|---|---|---|---|---|
| | 10<br>1<br>1<br>1<br>1<br>1<br>1<br>1<br>1<br>1<br>1 | 0 | 0 | |
| | 8<br>0<br>0<br>0<br>0<br>0<br>0<br>0<br>0 | 8 | 8 | |
| | 17<br>1<br>1<br>1<br>1<br>1<br>1<br>1<br>1<br>1<br>1<br>1<br>1<br>1<br>1<br>1<br>0<br>0 | 2 | 2 | |

Passed all tests!

Correc t

Marks for this submission: 1.00/1.00.

Back to Course

**Question 1** | Correct Mark 1.00 out of 1.00

Given an array `nums` of size `n`, return *the majority element*.

The majority element is the element that appears more than `⌊n / 2⌋` times. You may assume that the majority element always exists in the array.

**Example 1:**

```
Input: nums = [3,2,3]
Output: 3
```

**Example 2:**

```
Input: nums = [2,2,1,1,1,2,2]
Output: 2
```

**Constraints:**

- `n == nums.length`
- `1 <= n <= 5 * 10^4`
- `-2^31 <= nums[i] <= 2^31 - 1`

**For example:**

| Input | Result |
|---|---|
| 3<br>3 2 3 | 3 |
| 7<br>2 2 1 1 1 2 2 | 2 |

**Answer:** (penalty regime: 0 %)

```c
#include <stdio.h>

int main() {
    int n;
    scanf("%d", &n);

    int nums[n];

    for (int i = 0; i < n; i++)
        scanf("%d", &nums[i]);

    int candidate = nums[0];
    int count = 1;

    for (int i = 1; i < n; i++) {
        if (nums[i] == candidate)
            count++;
        else
            count--;

        if (count == 0) {
            candidate = nums[i];
            count = 1;
        }
    }

    printf("%d", candidate);

    return 0;
}
```

| | Input | Expected | Got | |
|---|---|---|---|---|
| | 3<br>3 2 3 | 3 | 3 | |

Passed all tests!

Correc t

Marks for this submission: 1.00/1.00.

Back to Course

**Question 1** | Correct Mark 1.00 out of 1.00

**Problem Statement:**

Given a sorted array and a value x, the floor of x is the largest element in array smaller than or equal to x. Write divide and conquer algorithm to find floor of x.

**Input Format**

First Line Contains Integer n – Size of array

Next n lines Contains n numbers – Elements of an array

Last Line Contains Integer x – Value for x

**Output Format**

First Line Contains Integer – Floor value for x

**Answer:** (penalty regime: 0 %)

```c
#include <stdio.h>

int floorValue(int a[], int low, int high, int x) {
    if (low > high)
        return -1;

    int mid = (low + high) / 2;

    if (a[mid] == x)
        return a[mid];

    if (a[mid] < x) {
        int result = floorValue(a, mid + 1, high, x);

        if (result == -1)
            return a[mid];
        else
            return result;
    }

    return floorValue(a, low, mid - 1, x);
}

int main() {
    int n, x;
    scanf("%d", &n);

    int a[n];

    for (int i = 0; i < n; i++)
        scanf("%d", &a[i]);

    scanf("%d", &x);

    int result = floorValue(a, 0, n - 1, x);

    printf("%d", result);

    return 0;
}
```

| | Input | Expected | Got | |
|---|---|---|---|---|
| | 6<br>1<br>2<br>8<br>10<br>12<br>19<br>5 | 2 | 2 | |
| | 5<br>10<br>22<br>85<br>108<br>129<br>100 | 85 | 85 | |
| | 7<br>3<br>5<br>7<br>9<br>11<br>13<br>15<br>10 | 9 | 9 | |

Passed all tests!

Correc t

Marks for this submission: 1.00/1.00.

Back to Course

**Question 1** | Correct Mark 1.00 out of 1.00

**Problem Statement:**

Given a sorted array of integers say arr[] and a number x. Write a recursive program using divide and conquer strategy to check if there exist two elements in the array whose sum = x. If there exist such two elements then return the numbers, otherwise print as "No".

Note: Write a Divide and Conquer Solution

**Input Format**

First Line Contains Integer n – Size of array

Next n lines Contains n numbers – Elements of an array

Last Line Contains Integer x – Sum Value

**Output Format**

First Line Contains Integer – Element1

Second Line Contains Integer – Element2 (Element 1 and Elements 2 together sums to value "x")

**Answer:** (penalty regime: 0 %)

```c
#include <stdio.h>

int findPair(int a[], int left, int right, int x) {
    if (left >= right)
        return 0;

    int sum = a[left] + a[right];

    if (sum == x) {
        printf("%d\n", a[left]);
        printf("%d\n", a[right]);
        return 1;
    }

    if (sum < x)
        return findPair(a, left + 1, right, x);
    else
        return findPair(a, left, right - 1, x);
}

int main() {
    int n, x;

    scanf("%d", &n);

    int a[n];

    for (int i = 0; i < n; i++)
        scanf("%d", &a[i]);

    scanf("%d", &x);

    if (!findPair(a, 0, n - 1, x))
        printf("No");

    return 0;
}
```

| | Input | Expected | Got | |
|---|---|---|---|---|
| ✔ | 4<br>2<br>4<br>8<br>10<br>14 | 4<br>10 | 4<br>10 | ✔ |

| | Input | Expected | Got | |
|---|---|---|---|---|
| | 5<br>2<br>4<br>6<br>8<br>10<br>100 | No | No | |

Passed all tests!

Correc t

Marks for this submission: 1.00/1.00.

Back to Course

**Question 1** | Correct Mark 1.00 out of 1.00

Write a Program to Implement the Quick Sort Algorithm

Input Format:

The first line contains the no of elements in the list-n

The next n lines contain the elements.

Output:

Sorted list of elements

**For example:**

| Input | Result |
|---|---|
| 5<br>67 34 12 98 78 | 12 34 67 78 98 |

**Answer:**

```c
#include <stdio.h>

void quickSort(int a[], int low, int high) {
    if (low < high) {
        int i = low;
        int j = high;
        int pivot = a[low];

        while (i < j) {
            while (a[i] <= pivot && i < high)
                i++;

            while (a[j] > pivot)
                j--;

            if (i < j) {
                int temp = a[i];
                a[i] = a[j];
                a[j] = temp;
            }
        }

        // Place pivot in correct position
        int temp = a[low];
        a[low] = a[j];
        a[j] = temp;

        quickSort(a, low, j - 1);
        quickSort(a, j + 1, high);
    }
}

int main() {
    int n;
    scanf("%d", &n);

    int a[n];

    for (int i = 0; i < n; i++)
        scanf("%d", &a[i]);

    quickSort(a, 0, n - 1);

    for (int i = 0; i < n; i++)
        printf("%d ", a[i]);

    return 0;
}
```

| | Input | Expected | Got | |
|---|---|---|---|---|
| | 5<br>67 34 12 98 78 | 12 34 67 78 98 | 12 34 67 78 98 | |
| | 10<br>1 56 78 90 32 56 11 10 90 114 | 1 10 11 32 56 56 78 90 90 114 | 1 10 11 32 56 56 78 90 90 114 | |
| | 12<br>9 8 7 6 5 4 3 2 1 10 11 90 | 1 2 3 4 5 6 7 8 9 10 11 90 | 1 2 3 4 5 6 7 8 9 10 11 90 | |

Passed all tests!

Correct

Marks for this submission: 1.00/1.00.

Back to Course

**Question 1** | Correct Mark 10.00 out of 10.00

**Playing with Numbers:**

Ram and Sita are playing with numbers by giving puzzles to each other. Now it was Ram term, so he gave Sita a positive integer 'n' and two numbers 1 and 3. He asked her to find the possible ways by which the number n can be represented using 1 and 3.Write any e fficient algorithm to find the possible ways.

**Example 1:**

**Input:** 6

**Output:** 6

**Explanation:** There are 6 ways to 6 represent number with 1 and 3

1+1+1+1+1+1

3+3

1+1+1+3

1+1+3+1

1+3+1+1

3+1+1+1

**Input Format**

First Line contains the number n

**Output Format**

**Print: The number of possible ways 'n' can be represented using 1 and 3**

Sample Input

6

Sample Output

6

**Answer:** (penalty regime: 0 % )

Ace editor not ready. Perhaps reload page?

Falling back to raw text area.

```
#include <stdio.h>
#include <stdlib.h>

long long find_ways(int n) {
    if (n < 0) return 0;
    if (n == 0 || n == 1 || n == 2) return 1;

    long long *dp = (long long *)malloc((n + 1) * sizeof(long long));
    if (dp == NULL) {
        return -1;
    }
        dp[0] = 1;
    dp[1] = 1;
    dp[2] = 1;

    for (int i = 3; i <= n; i++) {
        dp[i] = dp[i - 1] + dp[i - 3];
    }
```

| | Input | Expected | Got | |
|---|---|---|---|---|
| | 6 | 6 | 6 | |
| | 25 | 8641 | 8641 | |
| | 100 | 24382819596721629 | 24382819596721629 | |

Passed all tests!

Correc t

Marks for this submission: 10.00/10.00.

Back to Course

**Question 1** | Correct Mark 10.00 out of 10.00

**Playing with Chessboard:**

Ram is given with an n*n chessboard with each cell with a monetary value. Ram stands at the (0,0), that the position of the top left white rook. He is been given a task to reach the bottom right black rook position (n-1, n-1) constrained that he needs to reach the position by traveling the maximum monetary path under the condition that he can only travel one step right or one step down the board. Help ram to achieve it by providing an efficient DP algorithm.

**Example:**

**Input**

3

**1** 2 4

**2** 3 4

**8 7 1**

**Output:**

19

**Explanation:**

Totally there will be 6 paths among that the optimal is

Optimal path value:1+2+8+7+1=19

**Input Format**

First Line contains the integer n

The next n lines contain the n*n chessboard values

**Output Format**

Print Maximum monetary value of the path

**Answer:** (penalty regime: 0 %)

```c
#include <stdio.h>
#include <stdlib.h>

int main() {
    int n;
    if (scanf("%d", &n) != 1) return 0;

    long long **board = (long long **)malloc(n * sizeof(long long *));
    long long **dp = (long long **)malloc(n * sizeof(long long *));
    for (int i = 0; i < n; i++) {
        board[i] = (long long *)malloc(n * sizeof(long long));
        dp[i] = (long long *)malloc(n * sizeof(long long));
    }

    for (int i = 0; i < n; i++) {
        for (int j = 0; j < n; j++) {
            scanf("%lld", &board[i][j]);
        }
    }

    dp[0][0] = board[0][0];

    for (int j = 1; j < n; j++) {
        dp[0][j] = dp[0][j - 1] + board[0][j];
    }

    for (int i = 1; i < n; i++) {
        dp[i][0] = dp[i - 1][0] + board[i][0];
    }

    for (int i = 1; i < n; i++) {
        for (int j = 1; j < n; j++) {
            long long max_prev = (dp[i - 1][j] > dp[i][j - 1]) ? dp[i - 1][j] : dp[i][j - 1];
            dp[i][j] = board[i][j] + max_prev;
        }
    }
```

```
37
38      printf("%lld\n", dp[n - 1][n - 1]);
39
40      for (int i = 0; i < n; i++) {
41          free(board[i]);
42          free(dp[i]);
43      }
44      free(board);
45      free(dp);
46
47      return 0;
48  }
```

| | Input | Expected | Got | |
|---|---|---|---|---|
| ✔ | 3<br>1 2 4<br>2 3 4<br>8 7 1 | 19 | 19 | ✔ |
| ✔ | 3<br>1 3 1<br>1 5 1<br>4 2 1 | 12 | 12 | ✔ |
| ✔ | 4<br>1 1 3 4<br>1 5 7 8<br>2 3 4 6<br>1 6 9 0 | 28 | 28 | ✔ |

Passed all tests! ✔

Correct

Marks for this submission: 10.00/10.00.

Back to Course

**Question 1** | Correct Mark 1.00 out of 1.00

Given two strings find the length of the common longest subsequence(need not be contiguous) between the two.

Example:

s1: ggtabe

s2: tgatasb

| | | | | | | | |
|---|---|---|---|---|---|---|---|
| s1 | | a | g | **g** | **t** | **a** | **b** |
| s2 | **g** | x | **t** | x | **a** | y | **b** |

**The length is 4**

Solveing it using Dynamic Programming

**For example:**

| Input | Result |
|---|---|
| aab<br>azb | 2 |

**Answer:** (penalty regime: 0 %)

```c
#include <stdio.h>
#include <string.h>
#include <stdlib.h>

int max(int a, int b) {
    return (a > b) ? a : b;
}

int longestCommonSubsequence(char* s1, char* s2) {
    int m = strlen(s1);
    int n = strlen(s2);

    int **dp = (int **)malloc((m + 1) * sizeof(int *));
    for (int i = 0; i <= m; i++) {
        dp[i] = (int *)malloc((n + 1) * sizeof(int));
    }

    for (int i = 0; i <= m; i++) {
        for (int j = 0; j <= n; j++) {
            if (i == 0 || j == 0) {
                dp[i][j] = 0;
            }
            else if (s1[i - 1] == s2[j - 1]) {
                dp[i][j] = dp[i - 1][j - 1] + 1;
            }
            else {
                dp[i][j] = max(dp[i - 1][j], dp[i][j - 1]);
            }
        }
    }

    int result = dp[m][n];

    for (int i = 0; i <= m; i++) {
        free(dp[i]);
    }
    free(dp);

    return result;
}

int main() {
    char s1[1000], s2[1000];
```

```
44
45      if (scanf("%s %s", s1, s2) == 2) {
46          int lcs_length = longestCommonSubsequence(s1, s2);
47          printf("%d\n", lcs_length);
48      }
49
50      return 0;
51  }
```

| | Input | Expected | Got | |
|---|---|---|---|---|
| ✔ | aab<br>azb | 2 | 2 | ✔ |
| ✔ | ABCD<br>ABCD | 4 | 4 | ✔ |

Passed all tests! ✔

Correct

Marks for this submission: 1.00/1.00.

Back to Course

**Question 1** | Correct Mark 1.00 out of 1.00

Problem statement:

Find the length of the Longest Non-decreasing Subsequence in a given Sequence.

Eg:

Input:9

Sequence:[-1,3,4,5,2,2,2,2,3]

the subsequence is [-1,2,2,2,2,3]

Output:6

**Answer:** (penalty regime: 0 %)

```c
#include <stdio.h>
#include <stdlib.h>

int longestNonDecreasingSubsequence(int* arr, int n) {
    if (n == 0) return 0;

    int *dp = (int *)malloc(n * sizeof(int));
    int max_len = 1;

    for (int i = 0; i < n; i++) {
        dp[i] = 1;
    }

    for (int i = 1; i < n; i++) {
        for (int j = 0; j < i; j++) {
            if (arr[i] >= arr[j] && dp[j] + 1 > dp[i]) {
                dp[i] = dp[j] + 1;
            }
        }
        if (dp[i] > max_len) {
            max_len = dp[i];
        }
    }

    free(dp);
    return max_len;
}

int main() {
    int n;
    if (scanf("%d", &n) != 1) return 0;

    int *arr = (int *)malloc(n * sizeof(int));
    for (int i = 0; i < n; i++) {
        scanf("%d", &arr[i]);
    }

    int result = longestNonDecreasingSubsequence(arr, n);
    printf("%d\n", result);

    free(arr);
    return 0;
}
```

| | Input | Expected | Got | |
|---|---|---|---|---|
| ✔ | 9<br>-1 3 4 5 2 2 2 2 3 | 6 | 6 | ✔ |
| ✔ | 7<br>1 2 2 4 5 7 6 | 6 | 6 | ✔ |

Passed all tests!

Correc t

Marks for this submission: 1.00/1.00.

Back to Course

**Question 1** | Correct Mark 1.00 out of 1.00

Find Duplicate in Array.

Given a read only array of n integers between 1 and n, find one number that repeats.

Input Format:

First Line - Number of elements

n Lines - n Elements

Output Format:

Element x - That is repeated

**For example:**

| Input | Result |
|---|---|
| 5<br>1 1 2 3 4 | 1 |

**Answer:** (penalty regime: 0 %)

```c
#include <stdio.h>
#include <stdlib.h>

int findDuplicate(const int* nums, int n) {
    int low = 1, high = n;

    while (low < high) {
        int mid = low + (high - low) / 2;
        int count = 0;
                for (int i = 0; i < n; i++) {
            if (nums[i] <= mid) {
                count++;
            }
        }
                if (count > mid) {
            high = mid;
        } else {
            low = mid + 1;
        }
    }

    return low;
}

int main() {
    int n;
    if (scanf("%d", &n) != 1) return 0;

    int *nums = (int *)malloc(n * sizeof(int));
    for (int i = 0; i < n; i++) {
        scanf("%d", &nums[i]);
    }

    int duplicate = findDuplicate(nums, n);
    printf("%d\n", duplicate);

    free(nums);
    return 0;
}
```

| | Input | Expected | Got | |
|---|---|---|---|---|
| ✔ | 11<br>10 9 7 6 5 1 2 3 8 4 7 | 7 | 7 | ✔ |

| | Input | Expected | Got | |
|---|---|---|---|---|
| | 5<br>1 2 3 4 4 | 4 | 4 | |
| | 5<br>1 1 2 3 4 | 1 | 1 | |

Passed all tests!

Correc t

Marks for this submission: 1.00/1.00.

Back to Course

**Question 1** | Correct Mark 1.00 out of 1.00

Find Duplicate in Array.

Given a read only array of n integers between 1 and n, find one number that repeats.

Input Format:

First Line - Number of elements

n Lines - n Elements

Output Format:

Element x - That is repeated

**For example:**

| Input | Result |
|---|---|
| 5<br>1 1 2 3 4 | 1 |

**Answer:** (penalty regime: 0 %)

```c
#include <stdio.h>
#include <stdlib.h>

int findDuplicate(const int* nums, int n) {
    int slow = nums[0];
    int fast = nums[0];

    do {
        slow = nums[slow];
        fast = nums[nums[fast]];
    } while (slow != fast);

    slow = nums[0];
    while (slow != fast) {
        slow = nums[slow];
        fast = nums[fast];
    }

    return slow;
}

int main() {
    int n;
    if (scanf("%d", &n) != 1) return 0;

    int *nums = (int *)malloc(n * sizeof(int));
    for (int i = 0; i < n; i++) {
        scanf("%d", &nums[i]);
    }

    int duplicate = findDuplicate(nums, n);
    printf("%d\n", duplicate);

    free(nums);
    return 0;
}
```

| | Input | Expected | Got | |
|---|---|---|---|---|
| ✔ | 11<br>10 9 7 6 5 1 2 3 8 4 7 | 7 | 7 | ✔ |
| ✔ | 5<br>1 2 3 4 4 | 4 | 4 | ✔ |

| | Input | Expected | Got | |
|---|---|---|---|---|
| | 5<br>1 1 2 3 4 | 1 | 1 | |

Passed all tests!

Correc t

Marks for this submission: 1.00/1.00.

Back to Course

**Question 1** | Correct Mark 1.00 out of 1.00

Find the intersection of two sorted arrays.

OR in other words,

Given 2 sorted arrays, find all the elements which occur in both the arrays.

Input Format

· The first line contains T, the number of test cases. Following T lines contain:

1. Line 1 contains N1, followed by N1 integers of the first array
2. Line 2 contains N2, followed by N2 integers of the second array

Output Format

The intersection of the arrays in a single line

Example

Input:

1

3 10 17 57

6 2 7 10 15 57 246

Output:

10 57

Input:

1

6 1 2 3 4 5 6

2 1 6

Output:

1 6

**For example:**

| Input | Result |
|---|---|
| 1<br>3 10 17 57<br>6<br>2 7 10 15 57 246 | 10 57 |

**Answer:** (penalty regime: 0 %)

```c
#include <stdio.h>
#include <stdlib.h>

void solveIntersection() {
    int n1;
    if (scanf("%d", &n1) != 1) return;

    int *arr1 = (int *)malloc(n1 * sizeof(int));
    for (int i = 0; i < n1; i++) {
        scanf("%d", &arr1[i]);
    }

    int n2;
    scanf("%d", &n2);

    int *arr2 = (int *)malloc(n2 * sizeof(int));
    for (int i = 0; i < n2; i++) {
        scanf("%d", &arr2[i]);
    }

    int i = 0, j = 0;
    int first_element = 1;

    while (i < n1 && j < n2) {
        if (arr1[i] == arr2[j]) {
            if (!first_element) {
                printf(" ");
            }
            printf("%d", arr1[i]);
```

```
            first_element = 0;
            i++;
            j++;
        } else if (arr1[i] < arr2[j]) {
            i++;
        } else {
            j++;
        }
    }
    printf("\n");

    free(arr1);
    free(arr2);
}

int main() {
    int t;
    if (scanf("%d", &t) != 1) return 0;

    while (t--) {
        solveIntersection();
    }

```

| | Input | Expected | Got | |
|---|---|---|---|---|
| ✔ | 1<br>3 10 17 57<br>6<br>2 7 10 15 57 246 | 10 57 | 10 57 | ✔ |
| ✔ | 1<br>6 1 2 3 4 5 6<br>2<br>1 6 | 1 6 | 1 6 | ✔ |

Passed all tests! ✔

Correct

Marks for this submission: 1.00/1.00.

Back to Course

**Question 1** | Correct Mark 1.00 out of 1.00

Find the intersection of two sorted arrays.

OR in other words,

Given 2 sorted arrays, find all the elements which occur in both the arrays.

Input Format

· The first line contains T, the number of test cases. Following T lines contain:

1. Line 1 contains N1, followed by N1 integers of the first array
2. Line 2 contains N2, followed by N2 integers of the second array

Output Format

The intersection of the arrays in a single line

Example

Input:

1

3 10 17 57

6 2 7 10 15 57 246

Output:

10 57

Input:

1

6 1 2 3 4 5 6

2 1 6

Output:

1 6

**For example:**

| Input | Result |
|---|---|
| 1<br>3 10 17 57<br>6<br>2 7 10 15 57 246 | 10 57 |

**Answer:** (penalty regime: 0 %)

```c
#include <stdio.h>
#include <stdlib.h>

void solveIntersection() {
    int n1;
    if (scanf("%d", &n1) != 1) return;

    int *arr1 = (int *)malloc(n1 * sizeof(int));
    for (int i = 0; i < n1; i++) {
        scanf("%d", &arr1[i]);
    }

    int n2;
    scanf("%d", &n2);

    int *arr2 = (int *)malloc(n2 * sizeof(int));
    for (int i = 0; i < n2; i++) {
        scanf("%d", &arr2[i]);
    }

    int i = 0, j = 0;
    int first_element = 1;

    while (i < n1 && j < n2) {
        if (arr1[i] == arr2[j]) {
            if (!first_element) {
                printf(" ");
            }
            printf("%d", arr1[i]);
```

```
            first_element = 0;
            i++;
            j++;
        } else if (arr1[i] < arr2[j]) {
            i++;
        } else {
            j++;
        }
    }
    printf("\n");

    free(arr1);
    free(arr2);
}

int main() {
    int t;
    if (scanf("%d", &t) != 1) return 0;

    while (t--) {
        solveIntersection();
    }

```

| | Input | Expected | Got | |
|---|---|---|---|---|
| ✔ | 1<br>3 10 17 57<br>6<br>2 7 10 15 57 246 | 10 57 | 10 57 | ✔ |
| ✔ | 1<br>6 1 2 3 4 5 6<br>2<br>1 6 | 1 6 | 1 6 | ✔ |

Passed all tests! ✔

Correct

Marks for this submission: 1.00/1.00.

Back to Course

**Question 1** | Correct Mark 1.00 out of 1.00

Given an array A of sorted integers and another non negative integer k, find if there exists 2 indices i and j such that A[j] - A[i] = k, i != j.

Input Format:

First Line n - Number of elements in an array

Next n Lines - N elements in the array

k - Non - Negative Integer

Output Format:

1 - If pair exists

0 - If no pair exists

Explanation for the given Sample Testcase:

YES as 5 - 1 = 4

So Return 1.

**For example:**

| Input | Result |
|---|---|
| 3<br>1 3 5<br>4 | 1 |

**Answer:** (penalty regime: 0 %)

```c
#include <stdio.h>
#include <stdlib.h>

int hasPairWithDifference(const int* A, int n, int k) {
    if (n < 2) return 0;

    if (k == 0) {
        for (int i = 0; i < n - 1; i++) {
            if (A[i] == A[i + 1]) return 1;
        }
        return 0;
    }

    int i = 0, j = 1;
    while (i < n && j < n) {
        int diff = A[j] - A[i];
        if (i != j && diff == k) {
            return 1;
        } else if (diff < k) {
            j++;
        } else {
            i++;
            if (i == j) {
                j++;
            }
        }
    }
    return 0;
}

int main() {
    int n;
    if (scanf("%d", &n) != 1) return 0;

    int *A = (int *)malloc(n * sizeof(int));
    for (int i = 0; i < n; i++) {
        scanf("%d", &A[i]);
    }
```

```
    int k;
    scanf("%d", &k);

    int result = hasPairWithDifference(A, n, k);
    printf("%d\n", result);

    free(A);
    return 0;
}
```

| | Input | Expected | Got | |
|---|---|---|---|---|
| ✔ | 3<br>1 3 5<br>4 | 1 | 1 | ✔ |
| ✔ | 10<br>1 4 6 8 12 14 15 20 21 25<br>1 | 1 | 1 | ✔ |
| ✔ | 10<br>1 2 3 5 11 14 16 24 28 29<br>0 | 0 | 0 | ✔ |
| ✔ | 10<br>0 2 3 7 13 14 15 20 24 25<br>10 | 1 | 1 | ✔ |

Passed all tests! ✔

Correct

Marks for this submission: 1.00/1.00.

Back to Course

**Question 1** | Correct Mark 1.00 out of 1.00

Given an array A of sorted integers and another non negative integer k, find if there exists 2 indices i and j such that A[j] - A[i] = k, i != j.

Input Format:

First Line n - Number of elements in an array

Next n Lines - N elements in the array

k - Non - Negative Integer

Output Format:

1 - If pair exists

0 - If no pair exists

Explanation for the given Sample Testcase:

YES as 5 - 1 = 4

So Return 1.

**For example:**

| Input | Result |
|---|---|
| 3<br>1 3 5<br>4 | 1 |

**Answer:** (penalty regime: 0 %)

```c
#include <stdio.h>
#include <stdlib.h>

int hasPairWithDifference(const int* A, int n, int k) {
    if (n < 2) return 0;

    if (k < 0) k = -k;

    int i = 0, j = 1;
    while (i < n && j < n) {
        if (i == j) {
            j++;
            continue;
        }

        int diff = A[j] - A[i];

        if (diff == k) {
            return 1;
        } else if (diff < k) {
            j++;
        } else {
            i++;
        }
    }

    return 0;
}

int main() {
    int n;
    if (scanf("%d", &n) != 1) return 0;

    int *A = (int *)malloc(n * sizeof(int));
    for (int i = 0; i < n; i++) {
        scanf("%d", &A[i]);
    }

    int k;
```

```
40      scanf("%d", &k);
41
42      int result = hasPairWithDifference(A, n, k);
43      printf("%d\n", result);
44
45      free(A);
46      return 0;
47  }
```

| | Input | Expected | Got | |
|---|---|---|---|---|
| ✔ | 3<br>1 3 5<br>4 | 1 | 1 | ✔ |
| ✔ | 10<br>1 4 6 8 12 14 15 20 21 25<br>1 | 1 | 1 | ✔ |
| ✔ | 10<br>1 2 3 5 11 14 16 24 28 29<br>0 | 0 | 0 | ✔ |
| ✔ | 10<br>0 2 3 7 13 14 15 20 24 25<br>10 | 1 | 1 | ✔ |

Passed all tests! ✔

Correct

Marks for this submission: 1.00/1.00.

Back to Course